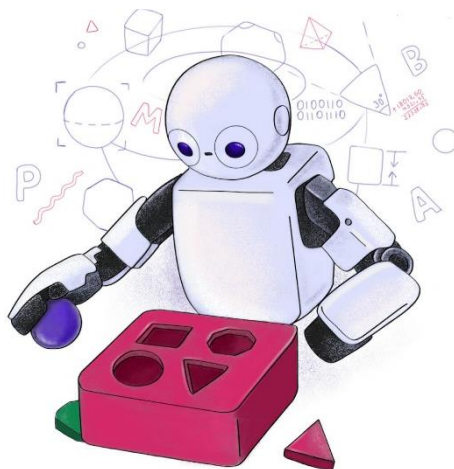
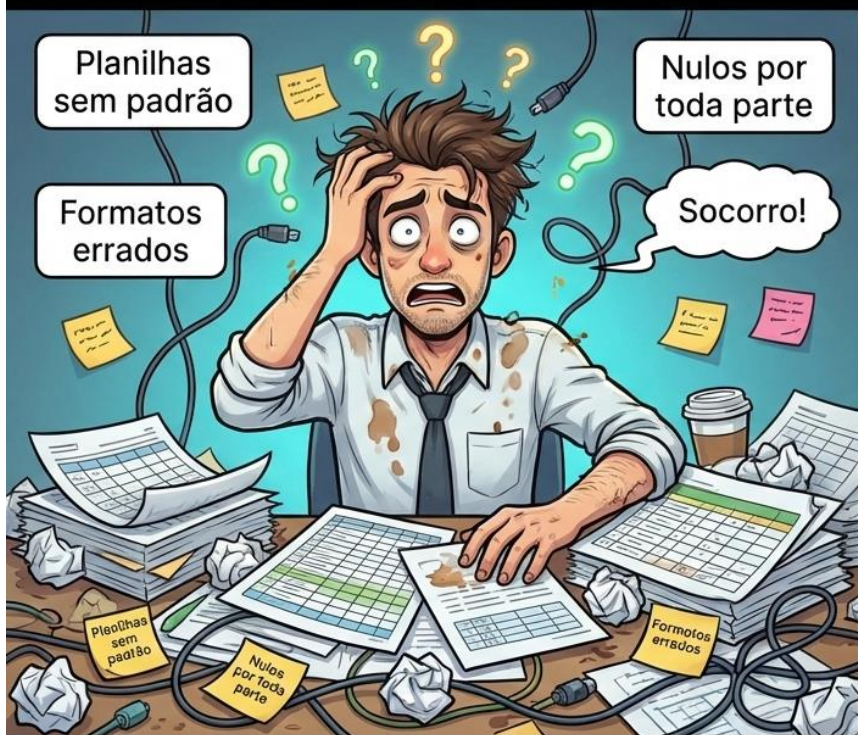


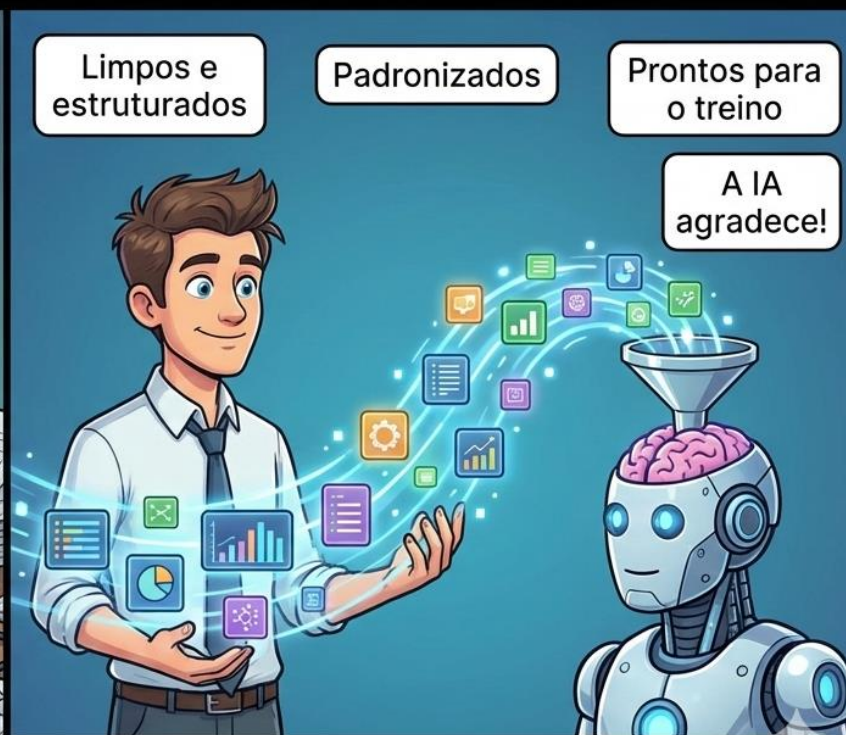
C24 - Inteligência Artificial: *Transformações dos dados*



DADOS BAGUNÇADOS: O caos que você conhece



DADOS PRONTOS PARA O MODELO: Música para os ouvidos da IA!



A jornada de todo cientista de dados: da bagunça à beleza

Continuamos na
jornada de
curadoria dos
dados.

O que veremos?

- Veremos algumas técnicas de transformação ou pré-processamento dos dados:
 - Divisão do *dataset*,
 - Escalonamento de atributos,
 - Codificação de atributos,
 - Engenharia de atributos,
 - Extração de atributos,
 - Seleção de atributos.

Antes de falarmos sobre as transformações, temos que falar sobre a divisão do *dataset* original

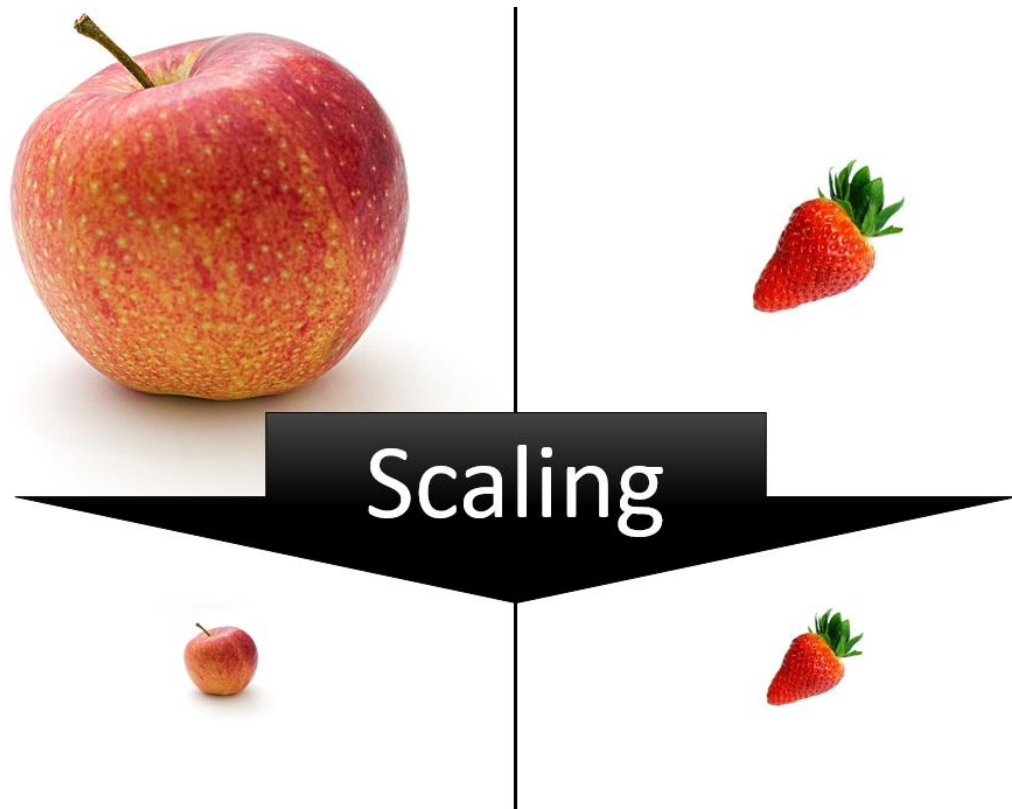
Divisão do *dataset*

- Em ML, o objetivo não é apenas aprender os dados disponíveis, mas generalizar bem para dados novos (i.e., inéditos).
- Para isso, o *dataset* é dividido em três subconjuntos:
 - Treinamento
 - Validação
 - Teste
- Todos os três devem ser representativos do problema/fenômeno sendo analisado.
- Em geral, mas não é regra, divide-se o *dataset* original da seguinte forma:
 - 70, 15, 15
 - 80, 10, 10

Divisão do *dataset*

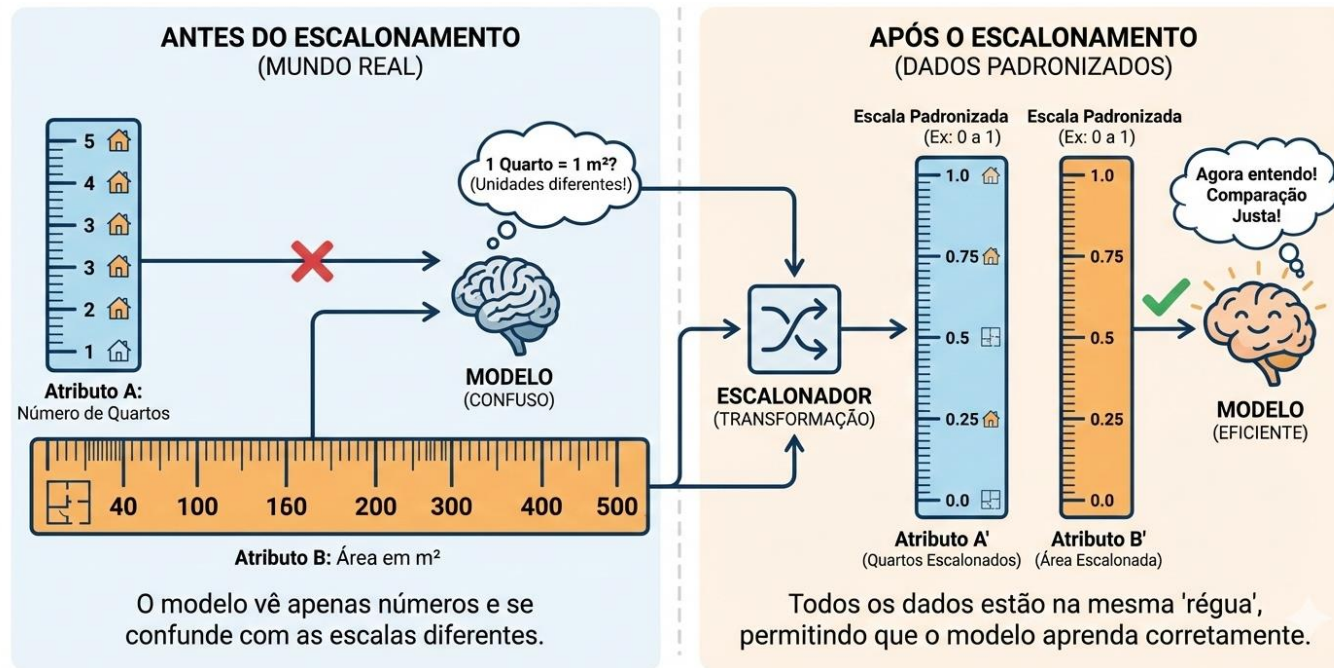
- O conjunto de treinamento é para treinar os modelos de ML.
 - É onde o modelo “aprende” os padrões dos dados.
 - Quanto mais representativo, melhor o aprendizado.
- O conjunto de validação avalia o modelo durante o desenvolvimento.
 - Evita que o modelo decore os dados de treinamento (i.e., se sobreajuste/*overfit*).
 - Ajustar hiperparâmetros.
 - Escolher arquiteturas.
 - Decidir quando parar o treinamento (*early stopping*).
- O conjunto de teste valida a capacidade de generalização dos modelos.
 - Usado uma única vez, ao final.
 - Fornece uma estimativa imparcial do desempenho real do modelo.
 - Simula dados nunca vistos.

Escalonamento de atributos



- Precisamos colocar todos os dados na mesma régua (i.e., **escala**).
- Imaginem criar um modelo para a previsão do preço de casas.
 - Atributo A: Número de quartos (1 a 5).
 - Atributo B: Área em m^2 (40 a 500).
- Para o modelo, 1 quarto é igual a 1 m^2 .
 - Os modelos não conhecem as unidades do mundo real, eles conhecem apenas números!

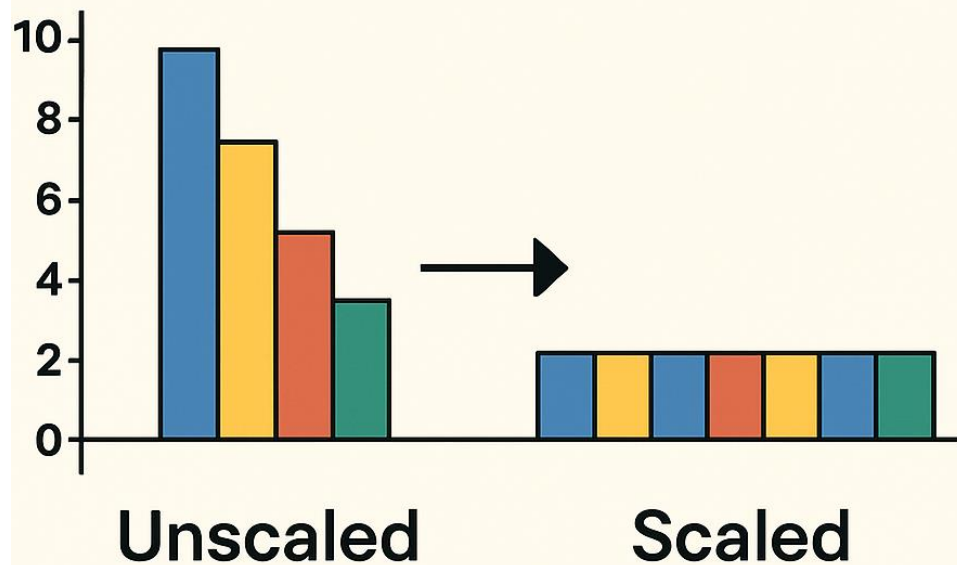
Escalonamento de atributos



- Precisamos colocar todos os dados na mesma régua (i.e., **escala**).
- Imaginem criar um modelo para a previsão do preço de casas.
 - Atributo A: Número de quartos (1 a 5).
 - Atributo B: Área em m^2 (40 a 500).
- Para o modelo, 1 quarto é igual a 1 m^2 .
 - Os modelos não conhecem as unidades do mundo real, eles conhecem apenas números!

Por que escalonar os atributos?

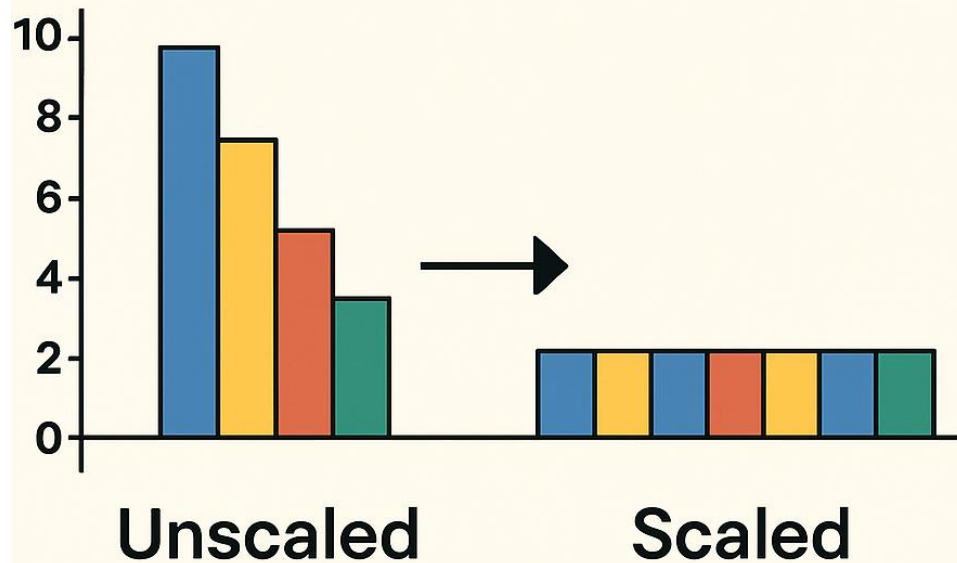
FEATURE SCALING



- Modelos de ML baseados em **distância** (e.g., KNN, SVM, PCA) ou **gradiente** (e.g., regressão linear/logística, redes neurais) são sensíveis à escala dos atributos.
- Atributos com escala muito maior influenciam muito mais a previsão dos modelos do que atributos com escala menor.
 - No exemplo da previsão do preço de casas, o modelo ignoraria os quartos e focaria só na área.

Por que escalonar os atributos?

FEATURE SCALING



- Mas e se todos os atributos são importantes para a previsão do modelo?
- O escalonamento iguala a influência dos atributos no modelo.
- Escalonamento transforma valores de atributos para uma mesma escala, geralmente sem unidade definida.
- Principais técnicas de escalonamento:
 - Padronização
 - Normalização
 - Escalonamento robusto

Normalização (Min-Max Scaling)

- **Como funciona:** Transforma todos os valores numéricos para uma escala fixa, geralmente entre 0 e 1.
- **Fórmula:**

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \in [0, 1],$$

onde x é o valor original, x_{min} e x_{max} são os valores mínimo e máximo de uma coluna (i.e., atributo).

Normalização (Min-Max Scaling)

- **Quando usar:**

- Dados sem *outliers* (analisar e remover antes),
- com algoritmos que não pressupõem nenhuma distribuição (e.g., redes neurais, KNN)
- e/ou que algoritmos que exigem entradas limitadas como os baseados em distância (e.g., KNN, SVM, K-Means) e gradiente (Redes Neurais, Regressões).

- **Cuidado:** Muito sensível a *outliers*. Um *outlier* extremo “esmaga” todos os outros valores para um intervalo muito pequeno próximo de zero.

- Os valores (que não são *outliers*) acabam sendo transformados em valores muito pequenos, próximos de zero, perdendo a distinção entre eles.
- Isso reduz a eficácia do aprendizado, pois a diferença entre os valores reais é praticamente eliminada na escala normalizada.

Normalização (Min-Max Scaling)

```
from sklearn.preprocessing import MinMaxScaler

# Instância do normalizador com intervalo de 0 a 1
scaler = MinMaxScaler(feature_range=(0, 1))

# Aprende os parâmetros de normalização no treino
X_scaled = scaler.fit_transform(X_train)

# Aplica o scaler aprendido no teste
X_test_scaled = scaler.transform(X_test)
```

Padronização (Standard Scaling)

- **Como funciona:** Transforma os atributos para terem média 0 e desvio padrão 1, não impondo limites.

- **Fórmula:**

$$x' = \frac{x - \mu_x}{\sigma_x},$$

onde x é o valor original, μ_x e σ_x são a média e o desvio padrão de uma coluna (i.e., atributo).

- **Vantagens:**

- Torna os dados "comparáveis" em termos do número de desvios padrão da média.
- Mais robusto a *outliers*, mas não a *outliers* extremos.
 - Por não dividir os valores por um denominador muito grande, no caso da presença de *outliers*, não os transforma em valores próximos de zero e, portanto, não se perde a distinção entre eles.

Padronização (Standard Scaling)

- **Quando usar:**
 - Em geral, com algoritmos que assumem distribuição "normal" ou Gaussiana (e.g., GNB, GMM, alguns modelos para detecção de anomalias), e
 - quando os dados possuem *outliers*.
- **Cuidado:** Se os dados originais são assimétricos, eles continuarão assimétricos, apenas centralizados e com variância igual a 1.
- **Importante:** Se o conjunto de dados tiver *outliers* extremos, o escalonador robusto é uma opção superior à padronização, pois ignora os valores discrepantes no cálculo do escalonamento.
 - *Outliers* podem influenciar a média e variância dos atributos de forma negativa.

Padronização (Standard Scaling)

```
from sklearn.preprocessing import StandardScaler
```

```
# Instância do padronizador
```

```
scaler = StandardScaler()
```

```
# Aprende os parâmetros de padronização no treino
```

```
X_scaled = scaler.fit_transform(X_train)
```

```
# Aplica o scaler aprendido no teste
```

```
X_test_scaled = scaler.transform(X_test)
```

Escalonamento robusto (Robust Scaling)

- **Como funciona:** Usa mediana e distância interquartil no escalonamento.
- **Fórmula:**

$$x' = \frac{x - \text{mediana}}{\underbrace{Q_3 - Q_1}_{\text{IQR}}},$$

onde x é o valor original e IQR é a distância interquartil de um atributo.

- **Vantagem:** Por utilizar a mediana e o IQR em vez da média e do desvio padrão, os *outliers* têm influência mínima nos dados escalonados.
 - Mantém a relação de distância entre os valores não-discrepantes melhor que a padronização quando existem *outliers* extremos.
 - Ou seja, não deixa que *outliers* extremos esmaguem os valores não-discrepantes em valores próximos de 0.

Escalonamento robusto (Robust Scaling)

- **Quando usar:**

- Quando há *outliers* significativos, mas desejamos mantê-los sem impactar o escalonamento.
- Quando as distribuições são assimétricas ou com caudas longas.

- **Cuidado:**

- Como não remove os *outliers*, algoritmos sensíveis à distância (e.g., KNN, Regressores) podem ter seu aprendizado impactado.
- A escala dos dados não discrepantes é mantida, mas ele não esconde os *outliers*.

Escalonamento robusto (Robust Scaling)

```
from sklearn.preprocessing import RobustScaler

# Instância do escalonador robusto
scaler = RobustScaler()

# Aprende os parâmetros de escalonamento no treino
X_scaled = scaler.fit_transform(X_train)

# Aplica o scaler aprendido no teste
X_test_scaled = scaler.transform(X_test)
```

Armadilhas comuns durante o escalonamento

- Para evitar vazamento de dados, o escalonador deve sempre ser ajustado no conjunto de treinamento.
- Os conjuntos de validação e teste, são transformados com os valores encontrados com o conjunto de treinamento.
- Geralmente, não se aplica escalonamento à variável alvo, i.e., os rótulos, apenas aos atributos.
- Não se deve escalonar atributos binários ou *one-hot* sem necessidade, pois pode-se criar valores fracionários desnecessários.

Codificação de atributos categóricos

- **Motivação:** Modelos matemáticos não entendem texto, portanto, precisamos converter categorias em números.
- As principais técnicas de codificação são:
 - One-Hot encoding
 - Label encoding
 - Target encoding
 - Binary encoding
- **OBS.:** A codificação pode ser considerada uma forma de engenharia de atributos.

One-Hot encoding

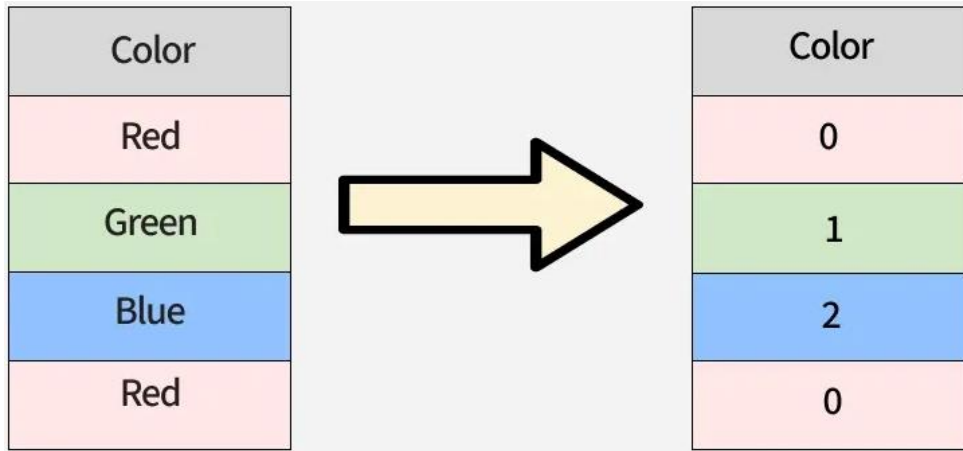


Color
Red
Red
Yellow
Green
Yellow

Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1
0	1	0

- Usada com atributos nominais (i.e., sem ordem) e com baixa cardinalidade (i.e., com poucas categorias distintas).
- **Como funciona:** Cria uma nova coluna binária para cada categoria distinta.
- **Limitação:** Gera alta dimensionalidade (i.e., muitas colunas) se houver muitas categorias, aumentando o custo computacional.
 - Pode criar o problema conhecido como a “maldição da dimensionalidade”.

Label encoding



- Converte cada categoria em um número inteiro único, mas sem considerar uma ordem.
- **Uso:** Indicada principalmente para a codificação do rótulo em problemas de classificação.
- **Risco:** Pode introduzir uma relação de ordem falsa que afeta negativamente algoritmos sensíveis a distâncias ou magnitudes (e.g., regressores linear e logístico e KNN).
 - Para tamanho de camisetas (P, M, G), faz sentido (ordinal).
 - Para animais e cores, não (nominal).

Target encoding

workclass	target		workclass	target mean		workclass
State-gov	0		State-gov	0		0
Self-emp-not-inc	1		Self-emp-not-inc	1		1
Private	0		Private	1/3		1/3
Private	0					1/3
Private	1					1/3

- Substitui cada categoria pela média do valor do alvo correspondente a essa categoria.
- **Quando usar:** Ideal para variáveis de alta cardinalidade, pois não aumenta a dimensionalidade do conjunto de dados e quando há relação entre a categoria e o *target*.
- **Risco:** *Overfitting* quando uma categoria tem poucos exemplos.
 - A média do *target* pode refletir mais ruído do que padrão real dos dados, prejudicando a generalização.

Binary encoding

Color		Color_0	Color_1
Green	Binary encoding →	0	0
Red		0	1
Black		1	0
Orange		1	1

- Combina os benefícios das codificações *One-Hot* e *Label*.
- Cada categoria recebe um número inteiro (*Label*), e depois esse número é convertido para sua representação binária, criando colunas para cada bit.
- **Quando usar:** Ótimo para variáveis categóricas de alta cardinalidade.
- **Desvantagem:** Por algumas categorias compartilharem bits, modelos lineares podem achar que existe uma relação matemática entre elas. Modelos de árvore (e.g., DT, RF) lidam bem com isso.

Exemplos de *encoding*

```
from sklearn.preprocessing import OneHotEncoder, LabelEncoder,  
TargetEncoder  
  
# SciKit-Learn não implementa o codificador binário (pip install  
category_encoders)  
  
from category_encoders import BinaryEncoder  
  
  
# LabelEncoder(), TargetEncoder() ou BinaryEncoder()  
enc = OneHotEncoder()  
enc.fit_transform(X_train)  
  
  
enc.transform(X_test)  
enc.inverse_transform(X_test)
```

Engenharia de atributos

- **Definição:** É a transformação ou criação de novos atributos que sejam mais úteis e informativos.
- **Por que fazer:** Ajuda o modelo a enxergar relações que não estão óbvias.
- **Objetivo:** Melhorar a capacidade de predição (generalização) do modelo.
- **Exemplos:**
 - Data de nascimento → Idade, faixa etária.
 - Coordenadas GPS → Distância até certo ponto.
 - Preço e quantidade → Valor total da venda.
 - *Timestamp* → Hora, dia da semana, feriado?
 - Comprimento e largura → Área.
 - Interação entre atributos → Polinômios (e.g., $x'_1 = x_1x_2 + x_1^2x_2$)

Engenharia de atributos

```
import pandas as pd
```

```
# Exemplo de dados
```

```
df = pd.DataFrame({  
    "preco": [10, 20, 15],  
    "quantidade": [2, 1, 3],  
    "desconto": [0, 5, 0]})
```

```
# Feature Engineering
```

```
df["valor_total"] = df["preco"] * df["quantidade"]  
df["houve_desconto"] = (df["desconto"] > 0).astype(int)
```

Extração de atributos

- **Definição:** É o processo de **transformar** dados brutos em um conjunto menor de atributos significativos que podem ser processados de forma mais eficiente, mantendo as informações essenciais (perde parte da informação).
 - **Transforma os dados em uma representação mais eficiente.**
- **Por que fazer:**
 - Reduz a dimensionalidade,
 - Remove redundância,
 - Reduz os recursos computacionais necessários para armazenamento, processamento e treinamento,
 - Acelera o treinamento,
 - Melhora o desempenho do modelo, pois os dados são mais informativos e menos ruidosos.

Técnicas comuns para extração de atributos

- **Dados tabulares:** Análise de componentes principais (PCA) e análise de componentes independentes (ICA).
 - Criam novos atributos não correlacionados que capturam a variância máxima (PCA) ou a independência dos atributos (ICA).
- **Texto:** *Bag of words (BoW), Term Frequency-Inverse Document Frequency (TF-IDF), Word Embeddings.*
 - Transformam texto em representações numéricas (modelos de ML entendem números), variando de contagens simples de palavras, a pesos informativos, até vetores densos que capturam significado e contexto.

Técnicas comuns para extração de atributos

- **Imagem:** *Histogram of Oriented Gradients (HOG), Scale-Invariant Feature Transform (SIFT), Convolutional Neural Network (CNN).*
 - Transformam *pixels* brutos em representações numéricas que capturam padrões visuais relevantes, como bordas, texturas, cores e estruturas semânticas, para facilitar o aprendizado dos modelos.
- **Séries temporais (áudio):** *Mel-frequency cepstral coefficients (MFCCs), Transformada rápida de Fourier (FFT), Espectogramas, Transformada de Wavelet.*
 - Transformam os sinais em características que capturam propriedades espectrais ou de tempo-frequência.

WHAT IF I TOLD YOU

**PCA ALLOWS PROJECTING HIGH
DIMENSIONAL DATA INTO LOWER DIMENSIONS**

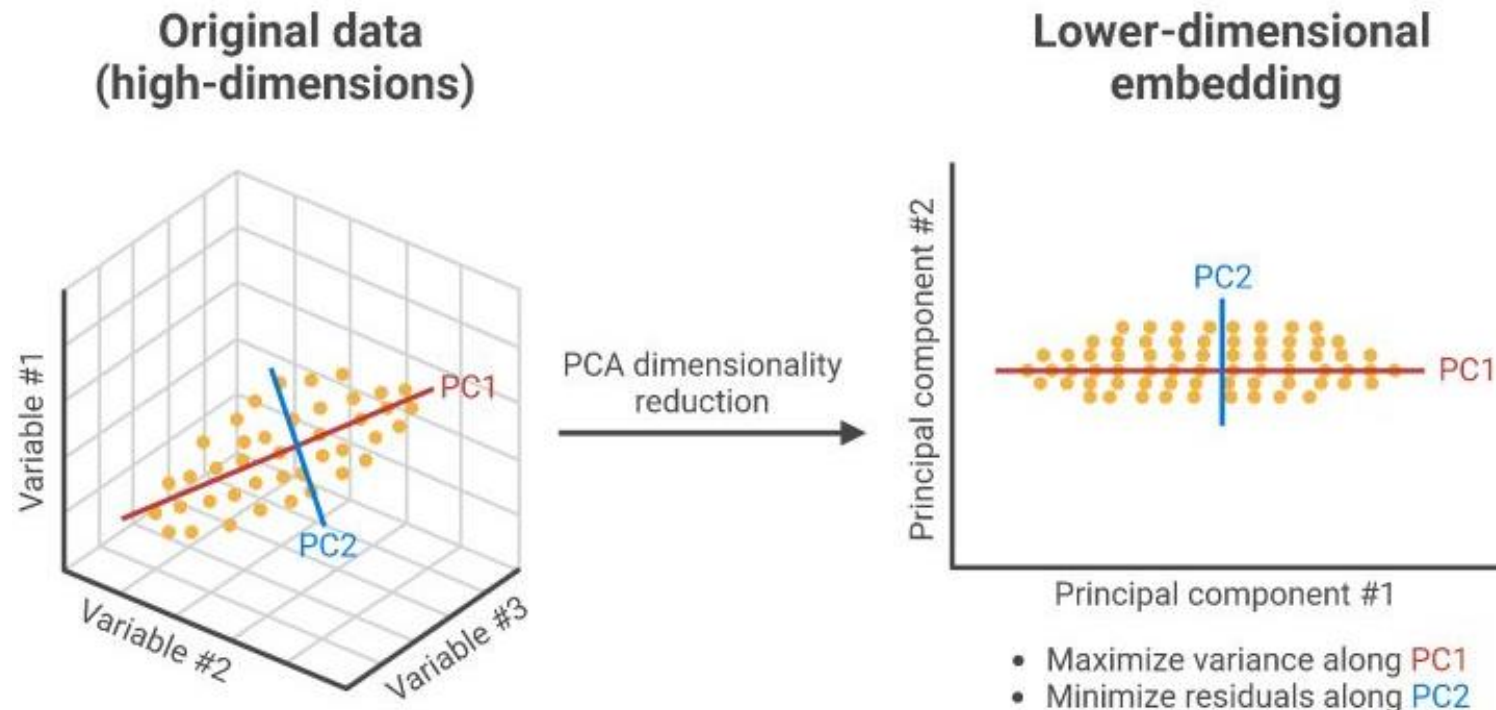


iStock™
Credit: meltonmedia

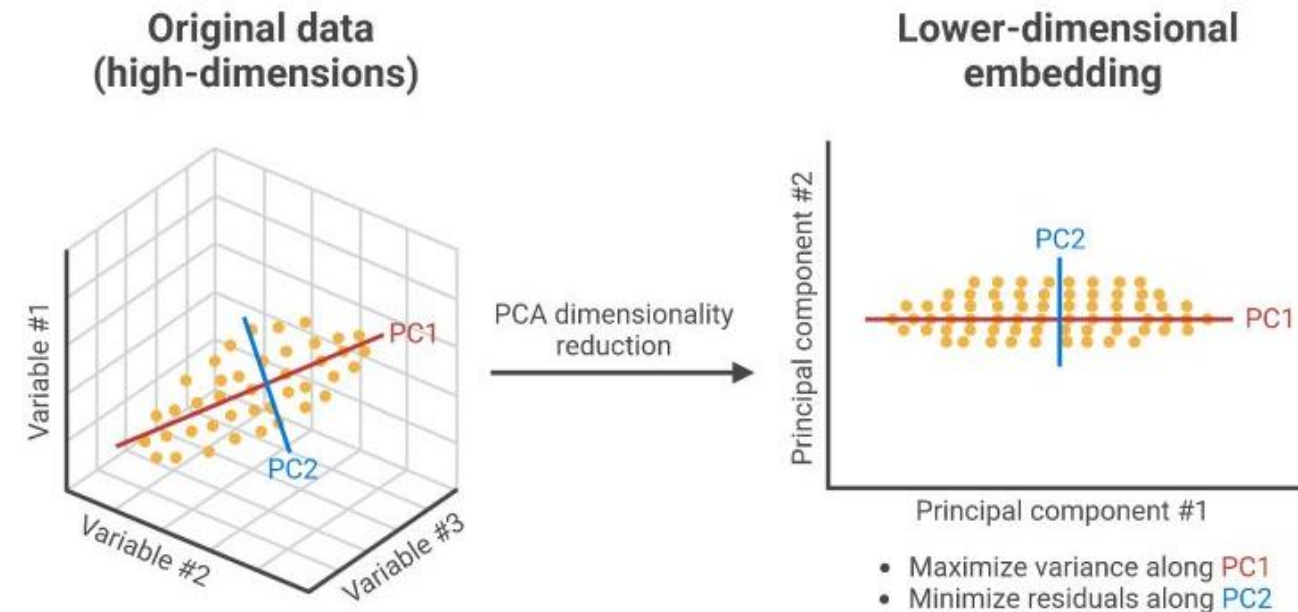
O PCA projeta dados de alta dimensão em um espaço de menor dimensão.

Análise de componentes principais (PCA)

- PCA funciona encontrando novas direções (componentes principais) que capturam o máximo de variância dos dados e **projetando** os dados nessas direções para reduzir a dimensionalidade com mínima perda de informação.



Análise de componentes principais (PCA)



- O algoritmo encontra as direções que melhor descrevem como os dados estão espalhados (i.e., máxima variância).
- Na figura, PC1 é a direção onde os dados variam mais.
- Já PC2 é a direção ortogonal à PC1 com a segunda maior variância.
- Mantendo PC1, preserva-se a maior parte da informação.
- PC2 representa variação residual (menor importância).

Análise de componentes principais (PCA)

- As PCs são ordenadas por variância.
 - A matriz de **projeção** é ordenada da componente de maior variância para a menor.
- Cada componente é uma **combinação linear** dos atributos originais.
- O PCA **assume relações lineares entre os atributos** (traça linhas retas para capturar as variâncias).
 - Se os dados originais seguem um padrão curvo (como uma senoide , meia lua ou um círculo), o PCA vai ter um desempenho bem ruim.
 - Usar técnicas não lineares como Kernel PCA e t-SNE.
- Deve ser aplicado após a separação do dados em conjuntos de treinamento, validação e teste para evitar vazamento.

Análise de componentes principais (PCA)

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

Escalonar os dados antes do PCA (é obrigatório que os atributos tenham a mesma variância)

```
X_scaled = StandardScaler().fit_transform(X)
```

Extrair 2 componentes

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_scaled)
```

```
print(pca.explained_variance_ratio_)
```

OBS.: “n_components” pode ser também um número entre 0 e 1 especificando o mínimo de variância explicada desejada.

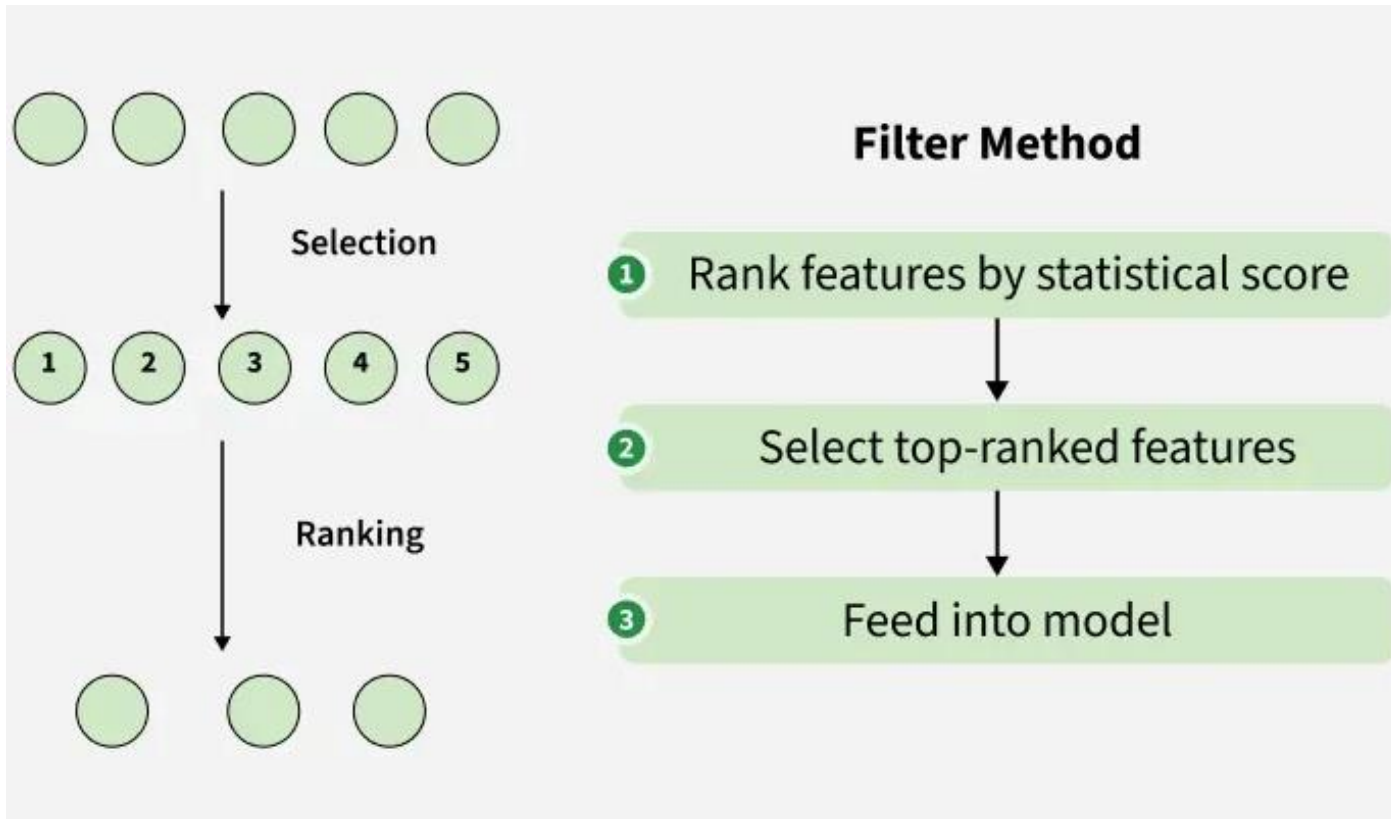
Seleção de atributos

- **Definição:** É o processo de selecionar um subconjunto de atributos que contribuem para a melhoria do desempenho do modelo.
- Por que usar?
 - Reduz *overfitting*: Modelos mais simples com menos atributos tendem a generalizar melhor para novos dados.
 - Reduz custo computacional: Menos dados significam algoritmos que processam mais rápido.
 - Facilita interpretação: É mais fácil explicar um modelo que utiliza 5 variáveis do que um que utiliza 500.

Seleção de atributos

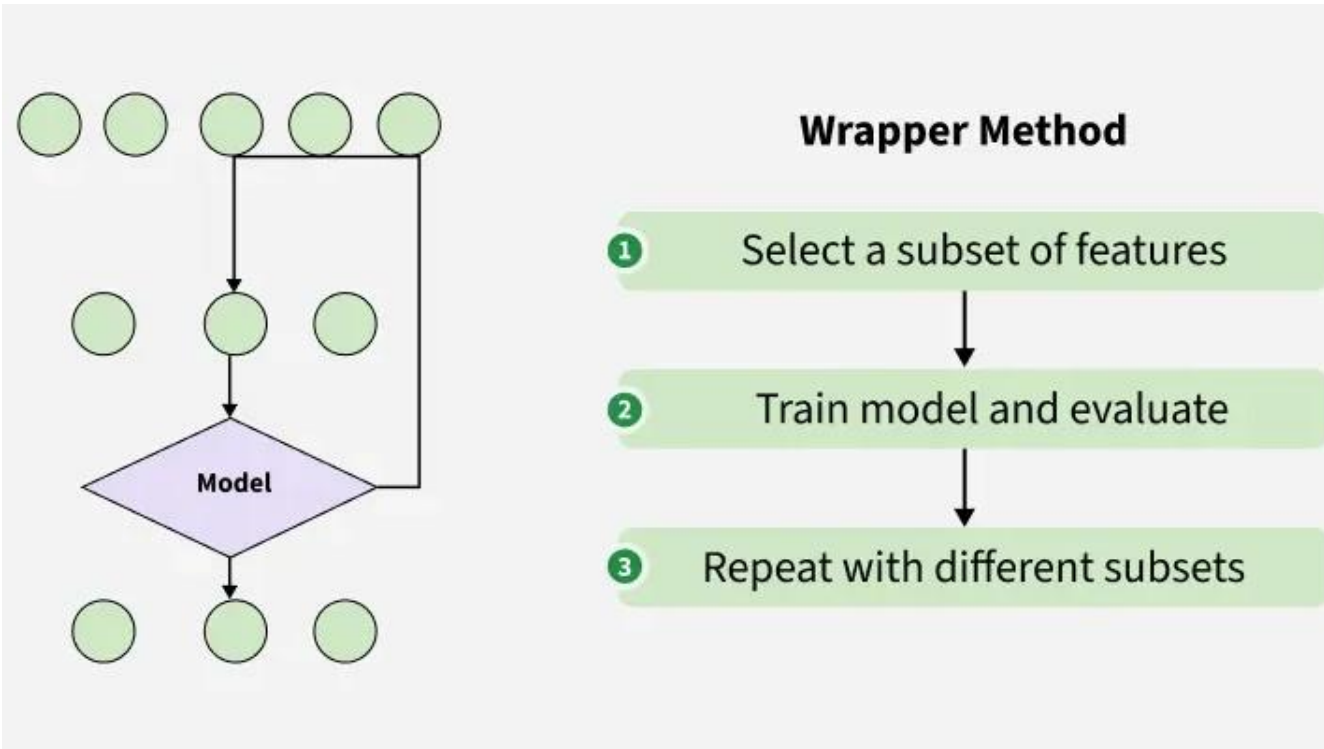
- Existem três tipos principais de técnicas de seleção de atributos:
 - **Filtragem:** Baseados em medidas estatísticas dos dados, como correlação.
 - **Exemplos:** Correlação de Pearson/Spearman, Teste qui-quadrado, limiar de variância, ganho de informação, informação mútua.
 - **Encapsulamento:** Usa o próprio modelo de ML para testar subconjuntos de variáveis e avaliar a performance.
 - **Exemplo:** Eliminação recursiva de atributos, .
 - **Embarcado:** A seleção ocorre durante o processo de treinamento do próprio algoritmo. É integrada (i.e., embarcada) no treinamento.
 - **Exemplos:** Regressão LASSO e árvores de decisão.

Métodos de filtro



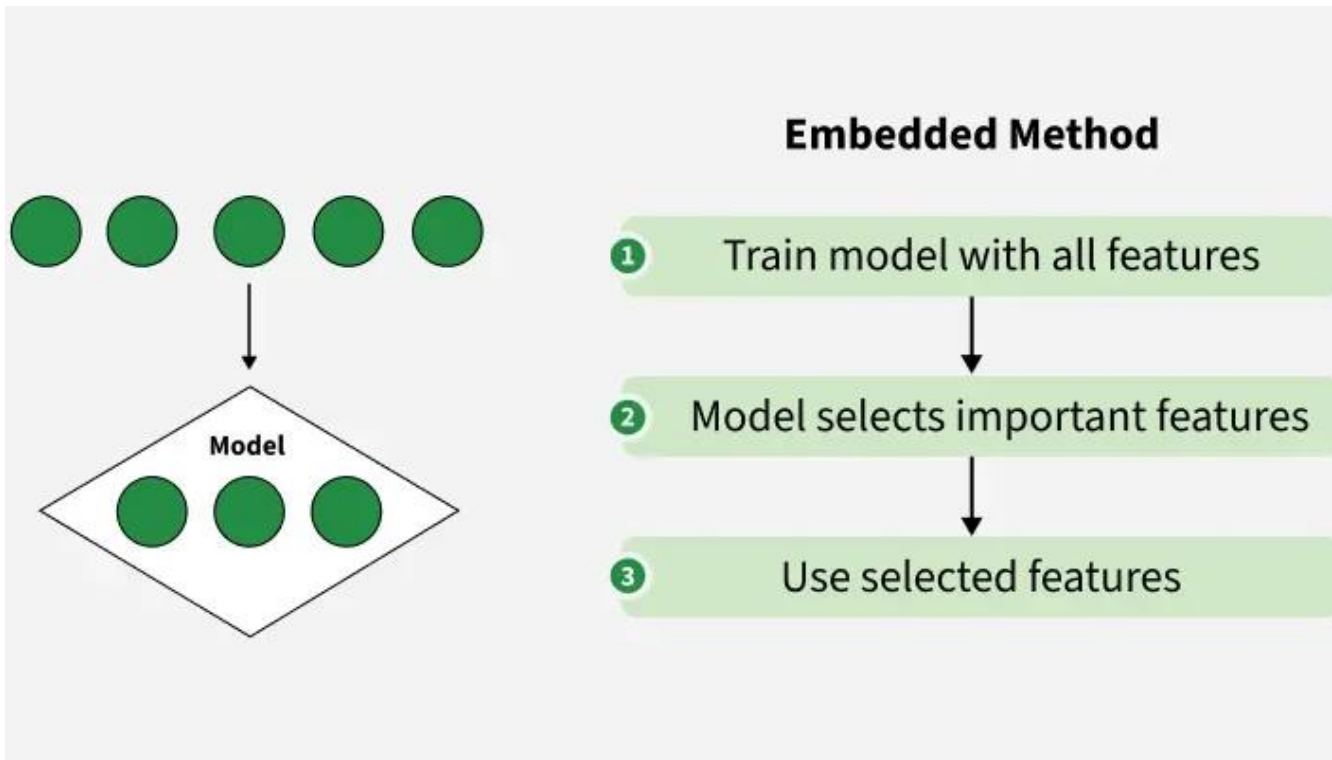
- São rápidos, simples e independentes do modelo, mas podem ignorar interações relevantes entre atributos (devido à independência do modelo) e dependem da escolha adequada da métrica (e.g., relações lineares e não lineares).

Métodos de encapsulamento



- Otimizam os atributos para um modelo específico e podem ter desempenho superior em comparação aos métodos de filtro, mas são computacionalmente caros e suscetíveis ao *overfitting*, pois podem ajustar excessivamente os atributos a um modelo específico.

Métodos embarcados



- Selecionam atributos durante o treinamento do modelo de forma eficiente, apresenta equilíbrio entre desempenho e tempo para seleção, mas têm interpretabilidade limitada e não são suportados por todos os algoritmos.

Seleção de atributos na biblioteca SciKit-learn

- O módulo `sklearn.feature_selection` implementa vários métodos de seleção de atributos, dentre eles estão:
 - Limiar de variância (`VarianceThreshold`)
 - Eliminação recursiva de atributos (`RFE`)
 - Selecione os K melhores atributos de acordo com um *score* (`SelectKBest`)
 - Seleção usando modelos com penalidade L1 ou baseados em árvore (`SelectFromModel`)
 - Seleção de atributos sequencial (`SequentialFeatureSelector`)

Seleção de atributos na biblioteca SciKit-learn

```
from sklearn.feature_selection import VarianceThreshold

# Remove features com variância <= 0.1
selector = VarianceThreshold(threshold=0.1)

X_reduced = selector.fit_transform(df)

# Quais features foram mantidas?
selected_features = df.columns[selector.get_support()]

print("\nFeatures selecionadas:")
print(selected_features)
```

Extração vs. seleção de atributos

- A extração **transforma** os atributos originais para criar novas, geralmente reduzindo a dimensionalidade.
- A seleção **escolhe** um subconjunto dos atributos existentes mais relevantes sem alterá-los.

Exemplo

- [Exemplo: pre_processing.ipynb](#)



Perguntas?

Obrigado!